



گزارش کار پروژه ربات با برد esp8266

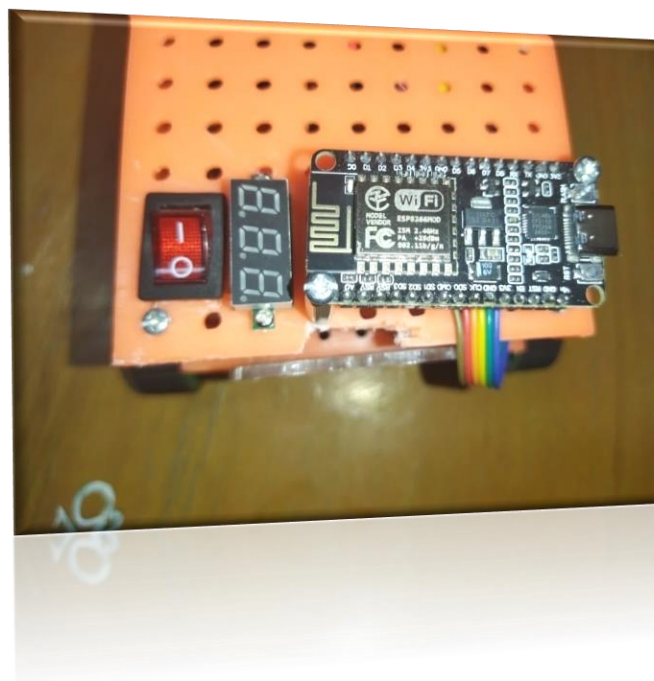
تهیه و تنظیم : مهدی نوروزی
دانشکده مهندسی برق دانشگاه قم

❖ برد esp8266

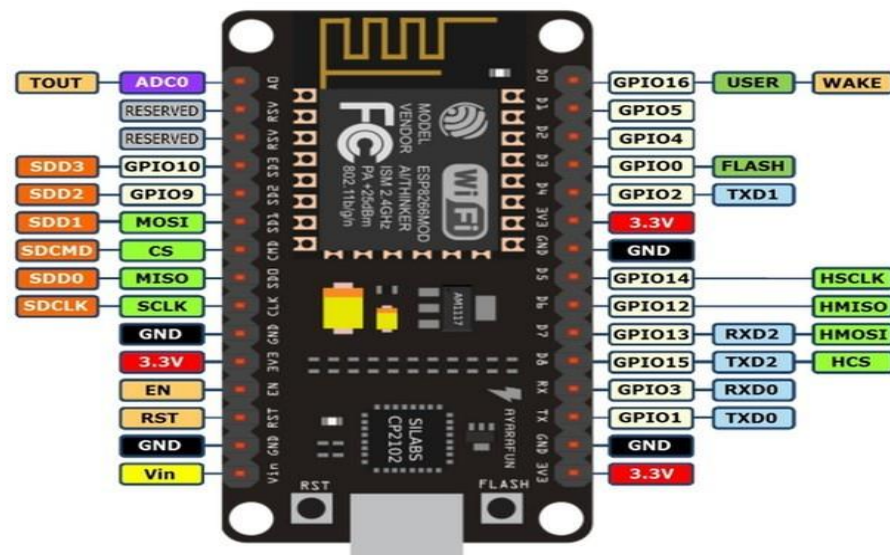
یک ماژول میکروکنترلری با قابلیت اتصال به شبکه‌های **Wi-Fi** است که به دلیل قیمت مناسب، اندازه کوچک و امکانات ارتباطی، در پروژه‌های اینترنت اشیا (IoT) و سیستم‌های کنترل از راه دور کاربرد زیادی دارد.

یکی از بردهای رایج مبتنی بر این تراشه، **NodeMCU ESP8266** است که دارای پایه‌های ورودی/خروجی دیجیتال، ورودی آنالوگ و رابط USB برای برنامه‌ریزی است. این برد را می‌توان با محیط‌هایی مانند **Arduino IDE** برنامه‌ریزی کرد و از طریق **Wi-Fi** برای کنترل تجهیزات به صورت بی‌سیم استفاده نمود.

در این پروژه، از **ESP8266** به عنوان واحد کنترل اصلی ربات و ایجاد ارتباط بی‌سیم با رایانه استفاده شده است



NodeMCU ESP12 Dev Kit V1.0 Pin Definition:

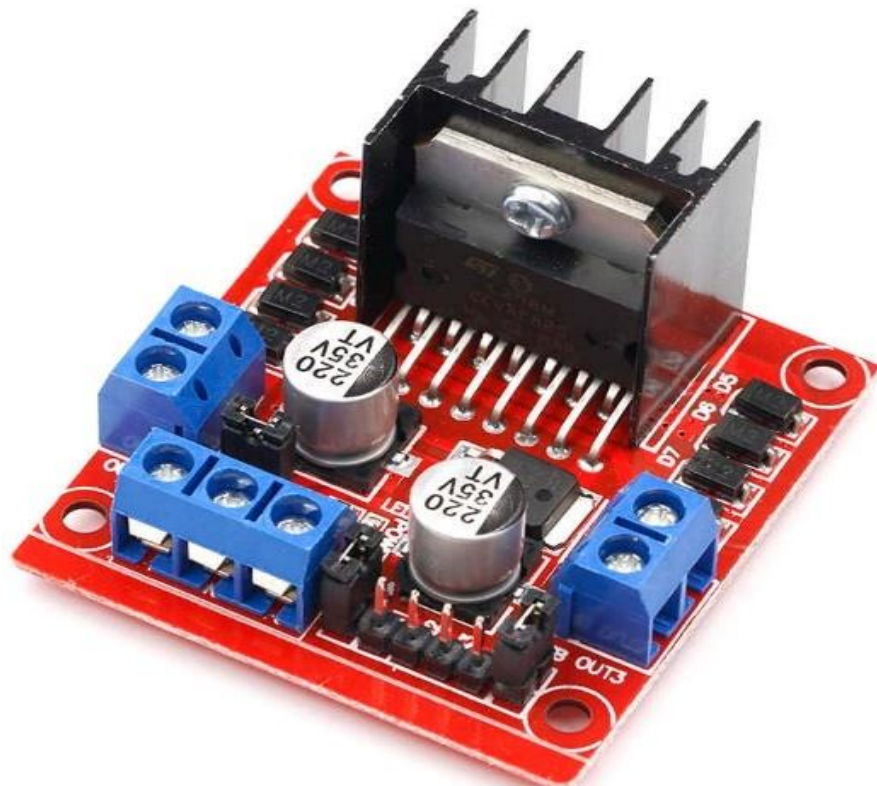


برد L298n

ماژول **L298N** یک درایور موتور دوکاناله است که برای کنترل موتورهای DC و موتورهای پله‌ای استفاده می‌شود. این ماژول امکان کنترل **جهت چرخش و سرعت موتور** را فراهم می‌کند و می‌تواند به صورت هم‌زمان دو موتور DC را کنترل کند.

L298N دارای دو کانال خروجی برای اتصال موتور ها و پایه های کنترلی **IN1**، **IN2**، **IN3** و **IN4** برای تعیین جهت چرخش است. همچنین پایه های **ENA** و **ENB** برای فعال سازی و کنترل سرعت موتور ها با استفاده از سیگنال PWM به کار می روند.

در این پروژه، از ماژول L298N برای راه اندازی و کنترل دو موتور DC ربات استفاده شده است. سیگنال های کنترلی از برد ESP8266 به L298N ارسال می شوند و درایور با تأمین جریان مورد نیاز موتور ها، جهت و سرعت حرکت آن ها را کنترل می کند.



❖ گزارشی از برنامه Arduino

1. بخش اول PIN CONFIGURATION

در این قسمت پایه های esp8266 به پایه های I298n متصل می شوند.

نکته : اگر بخواهیم ربات بدون تغییر سرعت حرکت کند (سرعت یکنواخت) کافی است از یک جامپر در ENA , ENB استفاده کنیم.

2. بخش دوم WI-FI CONFIGURATION

در این قسمت برای ربات یک شبکه وای فای تعریف میشود.
ربات در حالت access point کار میکند و لپ تاپ یا موبایل به آن متصل میشود و همچنین یک web server روی پورت 80 (http) ساخته میشود.

3. بخش سوم MOTOR SAFETY SETTINGS

در این قسمت تنظیم ایمنی موتور است و هدف آن جلوگیری از کنترل خشن و ناگهانی موتور است.
محدودیت PWM که حتی اگر max pwm ارسال شود باز ربات با $pwm = 500$ کار میکند. افزایش و کاهش تدریجی که در هر مرحله PWM به اندازه 8 افزایش (کاهش) می یابد و همچنین ناگهان PWM از 500 به 0 یا برعکس نمی رسد (هر مرحله 25 میلی ثانیه زمان میبرد).
تاخیر در تغییر جهت که اگر ربات بخواهد از جلو به عقب یا برعکس تغییر جهت دهد باید 250 میلی ثانیه متوقف شود این کار از تغییر جهت ناگهانی جلوگیری میکند.

4. بخش چهارم WI-FI SAFETY TIMEOUT

در این قسمت لپ تاپ هنگام حرکت مرتباً فرمان را به ربات می دهد و اگر ارتباط قطع شود ربات متوجه میشود که دیگر فرمانی دریافت نمی کند و اگر بیشتر از 700ms هیچ فرمان معتبری دریافت نشود ربات دستور توقف می گیرد.
مثلاً : اگر لپ تاپ قطع شود یا صفحه کنترل از کار بیفتد یا وای فای قطع شود نباید ربات به حرکت خود ادامه دهد.

5. بخش پنجم COMMAND / MOTOR STATE

در این بخش وضعیت حرکتی ربات تعریف میشود جلو. عقب. استاپ
همچنین توسط متغیر RobotDirection currentDirection وضعیت فعلی ربات مشخص است و توسط متغیر RobotDirection requestedDirection ربات متوجه میشود که

درخواست بعدی کاربر چیست و در این حالت برنامه فوراً موتور را معکوس نمیکند ابتدا توقف میکند

متغیر currentPWM مقدار pwm فعلی را حفظ میکند
3 متغیر آخر به ترتیب کنترل زمان شتاب گیری ، ارتباط وای فای و تاخیر تغییر جهت را استفاده میشود.

6. بخش ششم MOTOR DIRECTION

در این بخش به تعیین جهت موتور میپردازد و به موتور ها دستور میدهد که کار کنند یا توقف کنند ولی سرعت موتور را تنظیم نمی کند.

7. بخش هفتم APPLY PWM

در این بخش به کنترل سرعت 2 موتور می پردازد ابتدا PWM محدود میشود یعنی اگر اشتباهات PWM بیشتر از 600 باشد مقدار به 600 کاهش می یابد و بعد PWM به موتور ها اعمال میشود و سپس در یک متغیر currentPWM مقدار فعلی PWM ذخیره میشود

8. بخش هشتم EMERGENCY STOP

در این قسمت توقف اضطراری نرم افزار قرار داد وقتی که ربات بخواهد سریعاً به حالت سکون برود یعنی PWM به سرعت 0 میشود (ضریب 8 ندارد و با کمترین زمان صفر میشود)

9. بخش نهم SOFT STOP

در این قسمت توقف نرم ربات اتفاق میفتد و از PWM فعلی با کاهش تدریجی به صفر خواهیم رسید و وقتی به صفر برسیم جهت موتورها نیز خاموش میشود و همچنین زمان توقف در متغیر directionStopTime ذخیره میشود

10. بخش دهم START MOVEMENT

این قسمت مسئول شروع حرکت ربات است و برنامه اجازه نمیدهد که ربات مستقیماً جهت موتور را برعکس کند

حالت اول : ربات از قبل در همان جهت حرکت میکند و لازم نیست کاری انجام دهد

حالت دوم : ربات در جهت دیگری حرکت میکند در این حالت ربات متوقف میشود

حالت سوم : ربات متوقف است برنامه بررسی میکند اگر هنوز از 250 میلی ثانیه نگذشته باشد حرکتی انجام نمیدهد بعد از پایان زمان ایمنی جهت جدید اعمال میشود و PWM از صفر به صورت تدریجی افزایش میابد (افزایش سرعت تدریجی در بخش بعد است)

11. بخش یازدهم ACCELERATION

این تابع باعث میشود به تدریج ربات شتاب بگیرد .

اگر ربات متوقف باشد هیچ کاری انجام نمی دهد .

اگر ربات به حداکثر pwm رسیده باشد دیگر افزایش نمیدهد.

در غیر این 2 صورت هر 25 میلی ثانیه PWM به اندازه 8 واحد افزایش می یابد.

12. بخش دوازدهم PROCESS MOVEMENT

این تابع تقریباً مرکز تصمیم گیری ربات است این تابع دائماً بررسی میکند که وضعیت فعلی ربات چیست و کاربر چه چیزی درخواست کرده.

اگر ربات متوقف است (if (currentDirection == ROBOT_STOPPED) اگر فرمان جدیدی وجود دارد startMovement(requestedDirection);

اگر کاربر دستور استاپ داده باشد به صورت تدریجی ربات PWM به صفر میرسد

اگر جهت جدید متفاوت باشد برنامه این تابع softStop(); را اجرا میکند

اگر جهت تغییری نکرده باشد این تابع accelerate(); اجرا میشود

13. بخش سیزدهم HTML CONTROL PAGE

در این قسمت ظاهر صفحه سایت که در لپ تاپ دیده میشود که از css , javascript , html استفاده شده (کاملاً توسط AI نوشته شده)

14. بخش چهاردهم WEB SERVER: MAIN PAGE

مدیریت صفحه اصلی وب سرور

وقتی کاربر ادرس 192.168.4.1 را وارد میکند مرورگر یک ریکوئست به ربات ارسال میکند و برد این تابع را اجرا میکند server.send_P(...) سپس صفحه HTML را برای مرورگر ارسال میکند کد 200 نشانگر درخواست موفقیت آمیز است

15. بخش پانزدهم WEB SERVER: COMMAND HANDLER

در این قسمت دریافت فرمان از لپ تاپ اتفاق میفتد

مثلاً مرورگر درخواست هایی مانند

forward

Backward

Stop

ارسال میکند و ابتدا زمان آخرین تاریخ ارتباط معتبر ارسال میشود سپس برنامه صحت پارامتر
ارسالی را بررسی می کند که چنین فرمانی وجود دارد یا خیر (اگر وجود نداشت درخواست
نامعتبر اعلام میشود) 4 حالت وجود دارد :

UNKNOWN COMMAND / STOP / BACKWARD / FORWARD

16. بخش شانزدهم SETUP

راه اندازی اولیه که این تابع فقط یک بار در طول کار با ربات اجرا میشود یا همان مرحله آماده
سازی سیستم

مرحله اول : تنظیم پایه ها

ENA , ENB , In خروجی میشوند

مرحله دوم : وضعیت ایمن اولیه

قبل از اینکه وای فای وصل شود و وب سرور اجرا شود این کد emergencyStop و موتور در
وضعیت خاموش قرار میگیرد

مرحله سوم : سزیال

ارتباط سزیال با کامپیوتر اجرا میشود

مرحله چهارم : ایجاد access point

ESP در حالت access point قرار میگیرد و IP تعیین میشود و شبکه وای فای ساخته میشود

مرحله پنجم : ایجاد وب سرور

اگر کاربر صفحه اصلی را درخواست کرد handleRoot() اجرا میشود و فرمان ها از مسیر
/command توسط handleCommand() پردازش میشود سپس وب سرور ایجاد میشود

مرحله ششم : تنظیم تایمر ها

lastCommunication = millis();

lastRampTime = millis();

directionStopTime = millis();

تایمر های مورد نظر برای مقدار دهی اولیه هستند

Robot is READY. در پایان

MOTORS ARE STOPPED.

یعنی سیستم آماده دریافت فرمان است

17. بخش هفدهم MAIN LOOP

این تابع اصلی است که تکرار میشود

مرحله اول : دریافت درخواست ها

بررسی می کند که آیا لپ تاپ درخواست جدیدی فرستاده یا نه

مرحله دوم : بررسی timeout

اگر بیشتر از 700 میلی ثانیه ارتباطی نداشته باشد توقف کامل ربات

مرحله سوم : پردازش حرکت

این تابع processMovement() تصمیم میگیرد که ربات

حرکت کند؟ / شتاب بگیرد؟ / کاهش سرعت دهد؟ / متوقف شود؟ / تغییر جهت دهد؟

مرحله چهارم : yield

به esp اجازه میدهد که برنامه ها و وظایف داخلی خودش را نیز انجام دهد مخصوصا در قسمت وای فای بسیار مهم است